

1. Title of the Project

AI Real-Time Scam Link Detector

An AI-Powered URL Threat Intelligence System for Detecting Phishing and Scam Links on Messaging Platforms

2. Introduction

The rapid proliferation of smartphones and instant messaging platforms WhatsApp, Telegram, Instagram DMs, and SMS has transformed how people communicate and share information. However, this growth has also created a fertile ground for cybercriminals who exploit user trust through malicious links. Phishing URLs, scam websites, and malware distribution links are routinely embedded in messages that appear to originate from banks, delivery services, government agencies, or known contacts.

According to the Anti-Phishing Working Group (APWG), phishing attacks reached an all-time high in recent years, with over 4.7 million phishing attacks recorded globally. The financial impact is staggering the FBI Internet Crime Report estimates cybercrime losses in the billions of dollars annually, with phishing being one of the primary vectors.

Traditional defenses such as spam filters and blacklists suffer from a critical weakness: they are reactive. A malicious URL must be known and reported before it can be blocked. Attackers respond by registering new domains constantly, meaning a link may be active and harmful for hours or days before any blacklist catches it.

The mobile-first nature of modern communication compounds this problem. Mobile browsers and messaging apps provide less visual surface to inspect URLs, abbreviated previews hide suspicious domain names, and users on mobile devices are statistically more likely to click links without scrutiny. There is a clear need for an intelligent, proactive, real-time link-checking system that operates directly on the user's device.

This project the AI Real-Time Scam Link Detector addresses this gap by combining a machine learning phishing classifier with seven independent threat-intelligence modules, packaged as an Android mobile application backed by a containerized FastAPI service. Users can scan any URL in seconds by pasting it, sharing it from another app, or simply copying it to the clipboard, and receive a detailed, human-readable threat report with a 0-100 risk score and per-signal breakdown.

The system is built on modern, industry-standard technologies Python 3.10, FastAPI, Scikit-learn, Kotlin, and Docker making it maintainable, scalable, and deployable to any cloud infrastructure.

3. Problem Statement

Current Situation

Billions of potentially malicious links are shared every day through messaging platforms. Users have no reliable, fast, mobile-friendly mechanism to verify a URL's safety before clicking. Existing antivirus apps, browser safe-browsing features, and spam filters have the following shortcomings:

- Reactive Blacklists They only block URLs that have already been reported and catalogued. New phishing domains, which attackers register in minutes, evade detection entirely until the blacklist is updated.
- Single-Signal Decisions Many tools rely on one signal (e.g., Google Safe Browsing alone). A URL not yet in the database is classified as safe by default, creating a dangerous false sense of security.
- No Mobile Integration Desktop-centric security tools are not accessible when a user receives a suspicious link in WhatsApp and needs an answer in seconds.
- Lack of Explanation Even when a URL is flagged, users are rarely told why it is dangerous. Without understanding, users may bypass warnings or simply not trust the tool.
- Brand Impersonation Blind Spots Attackers register domains like paypa1.com or amazOn-offers.net that are visually convincing but are missed by pure keyword blacklists.

Core Problem

There is no accessible, real-time, multi-signal AI tool available to ordinary mobile users that can analyze a suspicious URL end-to-end and explain the specific threat indicators present.

Importance of Solving This Problem

- Directly reduces financial fraud and credential theft affecting millions of users.
 - Empowers non-technical users with actionable security intelligence.
 - Addresses phishing still the number-one entry vector for ransomware and data breaches.
-

4. Objectives of the Project

1. To design and develop a full-stack URL threat analysis system consisting of an Android mobile application and a cloud-deployable FastAPI backend.
 2. To implement a machine learning classifier trained on phishing and legitimate URL datasets to detect malicious URLs with high accuracy using 27+ structural URL features.
 3. To integrate multiple independent threat-intelligence modules domain age analysis, SSL certificate validation, brand impersonation detection, Google Safe Browsing, and redirect chain analysis and combine them into a single weighted risk score.
 4. To build a user-friendly Android application that allows users to scan URLs via paste, share intent, clipboard detection, and QR code scanning without requiring technical knowledge.
 5. To persist scan history locally and server-side, enabling users to review past scans and track threat trends over time.
 6. To produce transparent, explainable threat reports that detail each signal individually so users understand why a URL is rated as dangerous.
 7. To containerize and deploy the backend using Docker and Nginx so the system can be hosted on any cloud infrastructure with minimal configuration.
 8. To protect the API with rate limiting and API key authentication to prevent abuse and ensure service availability.
-

5. Scope of the Project

What the System Covers

- Real-time analysis of any HTTP or HTTPS URL submitted by a user.
- Seven-signal threat-intelligence pipeline producing a 0-100 composite risk score.
- Android mobile application with four scan input methods: manual paste, share intent, clipboard monitoring, and QR code scanning.
 - Persistent scan history accessible on both client (Room database) and server (SQLAlchemy/SQLite or PostgreSQL).
- REST API with full OpenAPI documentation, deployable via Docker Compose.

Target Users

- General mobile users who receive suspicious links via WhatsApp, Telegram, SMS, or email.
- Security-conscious individuals who want to verify links before sharing them.
- Small organizations that lack a dedicated security operations team and need a lightweight URL vetting tool.

Limitations

- The system analyzes the URL itself and associated metadata; it does not execute JavaScript or render the target webpage.
 - Domain WHOIS lookups may occasionally time out for certain TLDs with restricted WHOIS access.
 - Google Safe Browsing integration requires a valid API key and is subject to Google's usage quotas.
 - The ML model's accuracy is bounded by the quality and recency of its training dataset; periodic retraining is required.
 - The Android application requires Android 7.0 (API 24) or higher.
-

6. Literature Review

6.1 Existing Systems and Research

PhishTank and OpenPhish

PhishTank (phishtank.com) and OpenPhish are community-maintained URL blacklists. Users submit suspected phishing URLs, which are verified and added to a publicly queryable database. While widely used, these systems are entirely reactive – a URL must be reported, verified, and indexed before it offers any protection. Studies have shown that new phishing pages have an average lifespan of 4-8 hours, meaning many victims are compromised before any blacklist entry exists (Moore and Clayton, 2007).

Google Safe Browsing API

Google's Safe Browsing API cross-references URLs against Google's threat lists covering phishing, malware, and unwanted software. It is embedded in Chrome, Firefox, and Android's WebView. However, it suffers the same reactive limitation. Independent audits have found detection rates for zero-hour phishing URLs as low as 20-40% within the first hour of a campaign launching (Oest et al., 2020).

URLScan.io

URLScan.io renders the target webpage in a sandboxed browser and analyzes screenshots, DOM content, and network requests. This is highly effective but introduces significant latency of 10-30 seconds per scan and is not suitable for real-time mobile use. It also raises privacy concerns because the tool actually visits the URL on the user's behalf.

Machine Learning Approaches in Literature

Numerous academic works have applied machine learning to URL-based phishing detection:

- Sahingoz et al. (2019) evaluated seven ML classifiers on 73 URL features and found Random Forest achieved 97.98% accuracy, published in the Journal of Network and Computer Applications.
- Patil and Thepade (2019) proposed a hybrid feature extraction combining lexical and host-based features, achieving 96.3% accuracy with Gradient Boosting.
- Mohammad, Thabtah, and McCluskey (2014) introduced a widely cited phishing dataset of 11,055 URLs with 30 features, which remains a benchmark dataset in the domain.

VirusTotal

VirusTotal aggregates results from 70+ antivirus engines and URL scanners. It offers the most comprehensive coverage but requires API keys with strict rate limits on free tiers and has latency unsuitable for real-time mobile scanning.

6.2 Comparison of Available Solutions

Solution	Real-Time	Mobile App	ML-Based	Explainable	Free
PhishTank	No (blacklist)	No	No	No	Yes
Google Safe Browse	Partial	Via browser	No	No	Limited
VirusTotal	No (batch)	Partial	No	Partial	Limited
URLScan.io	No (slow)	No	No	Yes	Limited
This Project	Yes	Yes	Yes	Yes	Yes

6.3 Identified Gaps

- No existing free mobile application combines ML with multiple real-time threat-intelligence signals.
- Existing tools do not provide per-signal explanations accessible to non-technical users.
- Brand impersonation detection through fuzzy string matching is largely absent from popular consumer tools.
- Redirect chain analysis – uncovering the true final destination of a shortened or obfuscated URL – is not implemented in any free mobile client.

7. Proposed System

7.1 Overview

The proposed system is a two-component solution: an Android mobile application and a cloud-hosted FastAPI backend. The user submits a URL via the app; the backend runs it through a seven-stage analysis pipeline and returns a structured threat report in under 3 seconds.

7.2 Key Features

Mobile Application

- One-tap URL scanning from clipboard, share menu, or manual input.
- QR code scanning with integrated camera and ML Kit barcode decoder.
- Colour-coded threat verdict: Green (Safe) to Red (Dangerous).
- Per-signal breakdown cards showing domain age, SSL status, brand match result, and ML confidence.
- Persistent scan history with search and filter capability.

Backend API

- /scan endpoint accepts a URL and returns a full ThreatReport JSON object.
- Seven analysis modules run concurrently using Python async I/O for speed.
- Composite risk score (0-100) with five-tier verbal classification.
- /history endpoint for retrieving previous scan records.
- /health endpoint for uptime monitoring.

7.3 Improvements Over Existing Systems

- Proactive ML catches zero-hour phishing URLs not yet on any blacklist.
- Multi-signal fusion reduces false positives that single-signal tools produce.
- Mobile-native design with share-intent support for WhatsApp and Telegram.
- Transparent scoring every contributing signal is surfaced to the user.
- Open and self-hostable no vendor lock-in; deployable on any cloud VM.

8. Methodology

8.1 Development Model

The project follows an Iterative Agile development approach with four sprints covering backend core, threat-intelligence modules, Android application, and integration testing with deployment.

8.2 System Flow

User submits URL

v

Android App validates URL format

HTTPS POST /scan

v

FastAPI receives request > verifies API key > rate-limit check

+-- [Async] Extract 27+ URL features

+-- [Async] ML Phishing Classifier (30% weight)

+-- [Async] WHOIS Domain Age (12% weight)

+-- [Async] SSL Certificate Check (10% weight)

+-- [Async] Brand Impersonation (15% weight)

+-- [Async] Google Safe Browsing (13% weight)

+-- [Async] Redirect Chain Analysis (10% weight)

+-- [Sync] URL Feature Rules (10% weight)

v

Risk Engine combines weighted scores -> Final Score (0-100)

v

ThreatReport JSON returned to Android App

v

App renders colour-coded verdict + per-signal detail cards

Scan saved to Room (local) and ScanRecord table (server)

8.3 Machine Learning Model

- Algorithm: Random Forest Classifier (primary) with Gradient Boosting as alternative.
- Features: 27 URL features including length, special-character counts, digit ratio, subdomain depth, presence of IP address, HTTPS usage, URL entropy, suspicious keyword matching, TLD classification, and URL shortener detection.
- Training Data: UCI Phishing Websites Dataset plus supplementary APWG feeds totalling 11,000+ samples.
- Evaluation Metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC.
- Serialisation: Model saved as .joblib file and loaded once at application startup.

8.4 Testing Strategy

- Unit Testing: Each threat-intelligence module tested independently with known phishing and safe URLs.
- Integration Testing: Full /scan pipeline tested end-to-end against a labelled URL test set.
- Load Testing: Rate-limiter and async concurrency validated with multiple simultaneous requests.
- Android UI Testing: Espresso-based instrumentation tests for scan flow and history screen.
- Manual Testing: Real-world phishing URLs sourced from PhishTank used to validate live performance.

9. Technologies to Be Used

Layer	Technology	Version
Mobile Frontend	Kotlin	1.9+
Mobile UI Framework	Material Design 3	Jetpack
HTTP Client (Android)	Retrofit 2	2.9+
Local Database	Room	2.6+
Architecture Components	ViewModel + LiveData	Jetpack
QR Scanning	CameraX + ML Kit Barcode	Latest
Async (Android)	Kotlin Coroutines	1.7+
Backend Language	Python	3.10+
Web Framework	FastAPI	0.110+
ASGI Server	Uvicorn	0.27+
ML Library	Scikit-learn	1.4+
Data Processing	Pandas / NumPy	2.x / 1.x
Model Serialisation	Joblib	1.3+
Fuzzy Matching	RapidFuzz	3.x
WHOIS Lookups	python-whois	0.8+
Rate Limiting	SlowAPI	0.1+
Database	SQLite (dev) / PostgreSQL	Latest
ORM	SQLAlchemy (async)	2.x
Containerisation	Docker + Docker Compose	24+
Reverse Proxy	Nginx	1.25+
Backend Testing	Pytest	7.x+
Android Testing	Espresso	Latest
Platform (Server)	Ubuntu 22.04 LTS / Linux	LTS
Platform (Client)	Android 7.0+	API 24+

10. System Requirements

Hardware Requirements

Component	Minimum	Recommended
Server Processor	1 vCPU (1.0 GHz)	2 vCPU (2.4 GHz)

Component	Minimum	Recommended
Server RAM	1 GB	2 GB
Server Storage	10 GB SSD	20 GB SSD
Android Processor	ARMv7 1.2 GHz	ARMv8 / ARM64
Android RAM	2 GB	3 GB+
Android Storage	50 MB free	100 MB free
Network	3G or Wi-Fi	4G LTE or Wi-Fi

Software Requirements

Server / Development Environment

Requirement	Detail
Operating System	Ubuntu 22.04 LTS or Windows 10+
Programming Language	Python 3.10+
IDE	VS Code or PyCharm
Container Runtime	Docker 24+ with Docker Compose
Version Control	Git 2.40+
API Testing Tool	Postman or built-in Swagger UI

Android Development Environment

Requirement	Detail
IDE	Android Studio Hedgehog or newer
JDK	JDK 17
Build Tool	Gradle 8.x with Kotlin DSL
Android SDK	API 24 (minimum) API 34 (target)
Emulator or Device	Android 7.0 or higher

11. Expected Outcomes

1. A functional Android application that scans any URL and returns a threat verdict within 2-3 seconds on a standard 4G connection.
2. A REST API whose ML component achieves a measured 99.25% accuracy / 0.50% false-positive rate on a held-out test set of real phishing and legitimate URLs (see Section 11A for full methodology and confusion matrix). Reproducible via `python ml/train_model.py`.
3. Transparent threat reports listing each of the seven analysis signals individually, giving users and security teams actionable context.
4. Proactive zero-hour phishing detection through the ML component catching malicious URLs before they appear on any blacklist.
5. Reduced click-through rates on phishing links among users who adopt the application, contributing directly to reduced financial fraud and credential theft.
6. A reusable, open-architecture backend that any organization can self-host and extend with additional threat-intelligence modules.
7. Fully containerized deployment that can be reproduced on any Linux cloud VM in under 10 minutes using Docker Compose.

Benefits to Users

- Instant, free URL safety checks without technical knowledge required.
- Protection from financial scams, credential harvesting, and malware distribution.
- Peace of mind when receiving links from unknown senders on messaging platforms.

Industry Relevance

- Directly applicable to financial services, e-commerce platforms, and enterprise security teams as a URL pre-screening microservice.

- Aligns with the NIST Cybersecurity Framework (CSF) Detect and Protect functions.
- Suitable for integration into corporate mobile device management (MDM) pipelines.

11A. Experimental Results and Evaluation

All figures below are measured, not projected. They were produced by `ml/build_dataset.py` (data collection) and `ml/train_model.py` (training and evaluation), and are persisted verbatim in `ml/model/model_metadata.json`. They are reproducible end-to-end with two commands.

11A.1 Dataset

Property	Value
Phishing URLs	4,000 live feeds: OpenPhish + mitchellkrogza/Phishing.Database (active)
Legitimate URLs	4,000 top-100k popular domains (Tranco-style ranking)
Total samples	8,000 (balanced 50/50)
Features per sample	31 structural/lexical features (no page rendering, no network content)
Train / Test split	6,400 / 1,600 (stratified, random_state=42)
De-duplication	by host, to prevent near-duplicate leakage across the split

The previous version of this project trained on a synthetic dataset whose two classes were drawn from hand-separated distributions; it reported an AUC of 1.0, which is the mathematical fingerprint of a trivially separable, non-real problem. That dataset has been replaced entirely with the real-URL pipeline above. A regression test (`tests/test_predictor.py::test_metadata_metrics_are_honest`) now fails the build if AUC ever returns to exactly 1.0.

11A.2 Model Selection

Three classifiers were evaluated with stratified 5-fold cross-validation on the training split, scored by ROC-AUC. The best mean-CV model was selected and then evaluated once on the held-out test set (no test-set leakage in selection):

Model	CV ROC-AUC (mean ± std)
Random Forest	0.9994 ± 0.0006
Gradient Boosting (selected)	0.9996 ± 0.0002
Logistic Regression	0.9932 ± 0.0016

11A.3 Held-Out Test Performance (Real URLs)

Metric	Score
Accuracy	0.9925
Precision	0.9950
Recall	0.9900
F1-score	0.9925
ROC-AUC	0.9998

Confusion matrix (rows = true, columns = predicted; n = 1,600):

	Pred Safe	Pred Phishing
True Safe	796	4
True Phishing	8	792

False-positive rate = 4 / 800 = 0.50%; false-negative rate = 8 / 800 = 1.00%.

For a security screening tool the false-positive rate is the more important figure (legitimate sites wrongly blocked erode user trust), and it is kept below 1%.

11A.4 Feature Importance and Bias Control

The top discriminative features learned by the model are:

Rank	Feature	Importance
1	num_dots	0.65
2	num_digits_in_domain	0.13
3	domain_length	0.07
4	special_char_ratio	0.04
5	num_subdomains	0.03

num_dots and num_subdomains capture the classic subdomain-stuffing pattern (paypal.com.secure-login.attacker.tk) that is heavily documented in the phishing literature i.e. the model is keying on a genuine attack structure, not a corpus artifact.

****Two source biases were identified and explicitly corrected during dataset construction**** (documented in build_dataset.py):

1. IP-host shortcut raw feeds were ~45% bare-IP hosts whereas the legit list had none, letting the model "win" via has_ip_address alone. IP-host phishing is capped at 15% (MAX_IP_PHISHING_FRACTION).
2. Stale-HTTP shortcut the free feeds are ~96% HTTP, but modern phishing widely uses HTTPS (Let's Encrypt). ~80% of phishing URLs are upgraded to HTTPS (PHISHING_HTTPS_FRACTION) so has_https stops being a free separator.

After both corrections the headline accuracy is essentially unchanged (99.25%), which is evidence the result rests on genuine lexical/structural signal rather than a sampling artifact.

11A.5 Honest Limitations of These Numbers

- The legitimate set is drawn from popular domains; very long-tail legitimate URLs (new small businesses, dynamically generated links) are under-represented, so real-world false positives may exceed the 0.5% measured here.
- URLs are evaluated lexically (no page rendering). Phishing pages hosted on compromised legitimate domains, or behind cloaking, will not be caught by the ML signal alone this is precisely why the system fuses six additional signals and applies hard override floors.
- The 31-feature model is deliberately lightweight for sub-3-second mobile latency; a transformer-based content model (future work) would trade latency for recall on content-based attacks.

11A.6 Reproducibility

```
cd backend
python ml/build_dataset.py --phishing 4000 --legit 4000 # fetch real URLs
python ml/train_model.py                               # train + write metrics
pytest -q                                               # 24 tests incl. regressions
```

11B. Defence Relevance (DRDO Context)

While framed for general mobile users, the system's architecture maps directly onto defence cybersecurity priorities:

1. Counter-spear-phishing for personnel. Spear-phishing is the dominant initial-access vector in nation-state intrusion campaigns. An on-device, real-time link verifier is a practical control for defence personnel who receive links over personal and official messaging channels.
 2. Data sovereignty / air-gap friendliness. The backend is fully self-hostable with no mandatory third-party dependency. Google Safe Browsing is an optional signal that degrades gracefully when disabled. The ML + lexical + WHOIS + SSL signals run entirely on infrastructure the operator controls, so URLs from sensitive contexts need never leave a trusted network. This directly supports indigenous, no-foreign-cloud deployment.
 3. Offline / field deployment. The roadmap on-device ONNX model (Future Enhancement #8) enables partial analysis on disconnected or intermittently-connected devices, relevant to field conditions.
 4. Explainable, auditable decisions. Every verdict ships with a per-signal breakdown and override trail (`_component_scores`, `threat_summary`), enabling analyst review rather than opaque allow/deny aligned with defence-grade accountability requirements.
 5. Extensible threat-intel fusion. The weighted-fusion engine with hard override floors is a clean substrate for plugging in classified or subscription threat feeds (AbuseIPDB, AlienVault OTX, internal IOC lists) without re-architecting.
 6. Standards alignment. Maps to the NIST CSF Detect/Protect functions and is deployable as a URL pre-screening microservice inside an MDM pipeline.
-

12. Future Enhancements

1. Browser Extension. Port the URL scanning capability to a Chrome or Firefox extension for desktop users.
 2. iOS Application. Develop a Swift/SwiftUI counterpart to extend coverage to Apple devices.
 3. Real-Time Continuous Monitoring. Background service that monitors all links in the notification shade and alerts users before they open a browser.
 4. Advanced AI Model. Replace the Random Forest classifier with a fine-tuned transformer-based model (e.g., BERT) that analyzes page title, meta-description, and favicon similarity for richer detection.
 5. Threat Intelligence Feed Subscription. Integrate commercial threat feeds such as AlienVault OTX, Shodan, and AbuseIPDB for broader and more current coverage.
 6. Community Reporting. Allow users to report false negatives, contributing to a continuously improving training dataset.
 7. Organization Dashboard. Admin portal with aggregate analytics, team scan history, and policy-based URL blocking rules.
 8. Offline ML Mode. Compress and ship a lightweight on-device ONNX model for partial analysis without internet connectivity.
 9. Multi-Language Support. Localize the Android app for regional markets where phishing attacks in local languages are prevalent.
 10. Advanced Analytics. Trend detection to identify coordinated phishing campaigns targeting specific brands or regions.
-

13. Timeline (Project Schedule)

Phase	Activity	Duration
Phase 1	Requirement Analysis stakeholder interviews, existing system study, feature list finalization	7 days
Phase 2	System Design architecture design, database schema, API contract, UI wireframes	7 days
Phase 3	Backend Core Development FastAPI setup, database models, URL feature extractor, ML training	14 days
Phase 4	Threat Intelligence Modules WHOIS, SSL, brand impersonation, Safe Browsing, redirect checker	14 days
Phase 5	Android Application Development UI screens, Retrofit integration, Room database, scan history	14 days
Phase 6	Integration and System Testing end-to-end tests, load testing, UI testing, bug fixing	7 days
Phase 7	Deployment Docker Compose setup, Nginx config, cloud deployment, CI/CD pipeline	5 days
Phase 8	Documentation and Final Review README, API docs, synopsis, project report, presentation	4 days
Total		72 days

14. References

1. Sahingoz, O. K., Buber, E., Demir, O., and Diri, B. (2019). Machine learning based phishing detection from URLs. *Journal of Network and Computer Applications*, 127, 1-12. <https://doi.org/10.1016/j.jnca.2018.12.003>
2. Oest, A., Safaei, Y., Doupe, A., Ahn, G. J., Wardman, B., and Warner, G. (2020). PhishTime: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists. *Proceedings of the 29th USENIX Security Symposium*, 379-396.
3. Moore, T., and Clayton, R. (2007). Examining the impact of website take-down on phishing. *Proceedings of the Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit*. ACM. <https://doi.org/10.1145/1299015.1299016>
4. Mohammad, R. M., Thabtah, F., and McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443-458. <https://doi.org/10.1007/s00521-013-1490-z>
5. Federal Bureau of Investigation. (2023). Internet Crime Report 2023. Internet Crime Complaint Center (IC3). https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf
6. Anti-Phishing Working Group (APWG). (2023). Phishing Activity Trends Report, Q4 2023. <https://apwg.org/trendsreports/>
7. FastAPI Documentation. (2024). FastAPI Modern, fast web framework for building APIs with Python. <https://fastapi.tiangolo.com>
8. Scikit-learn Developers. (2024). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://scikit-learn.org>